

GLOBAL1D output conventions: reference and implementation guide

Anna Kelbert with Claude

September 2026

Abstract

GLOBAL1D has two independent 1D layered-sphere solvers, `field1d.f90` (S1) and `field1d_s2.f90` (S2). Each solves the same physics (a toroidal potential $T_l(r)$ driven by an external multipole source, propagated through a layered conductor) but the two were written 15 years apart, from different source references, and originally disagreed on how the raw numbers are bookkept: which time convention, which spherical-harmonic normalization, which way θ and r increase, and what a unit source coefficient physically means. `EARTH/FWD/output_convention.f90` resolves this with a **composable output-convention layer**: a small `output_convention_t` type describing seven bookkeeping dimensions, five named presets built from it, and a pair of transforms (one applied to the source coefficients before the solve, one applied to the finished field arrays after) that convert either solver’s native output into any named convention. Neither solver’s internal numerics are ever touched — every convention change is a rescale before the solve or an array transform after it. This document is the finalized reference for what each dimension and preset means, exactly what the transforms compute, and how to extend the system.

1 The seven bookkeeping dimensions

`output_convention_t` (`EARTH/FWD/output_convention.f90`) tracks:

#	Field	Values	Meaning
1	<code>time_convention</code>	PLUS_IWT / MINUS_IWT	$e^{+i\omega t}$ or $e^{-i\omega t}$
2	<code>harmonic_norm</code>	SCHMIDT_SEMINORM / FULLY_NORM	spherical-harmonic normalization
3	<code>condon_shortley</code>	.true. / .false.	Condon-Shortley $(-1)^m$ phase
4	<code>theta_convention</code>	COLAT_N2S / LAT_S2N	colatitude N→S, or latitude S→N
5	<code>r_convention</code>	R_DOWN / R_UP	radius increasing with depth, or with r
6	<code>primary_grid</code>	'H' / 'E'	which field is EDGE-staggered (bookkeeping only)
7	<code>radial_norm</code>	MULTIPOLE/SURFACE/ DIMENSIONAL/POTENTIAL	meaning of a unit source coefficient

Table 1: The seven convention dimensions tracked by `output_convention_t`.

Plus one orthogonal boolean, `modem_normalize`, which is not a bookkeeping choice but a value rescale (§4.4) — it does not change what a field component *represents*, only its absolute scale, so it lives outside the seven dimensions above.

Dimensions 1–3 are properties of the **input coefficients and the angular basis functions** they multiply; 4–5 are properties of the **output grid indexing and vector-component sign**; 6 records which of H/E is EDGE-primary for the active solver (never itself transformed — see §4.3); 7 is a property of the **radial amplitude scale** of the source.

Physical meaning of each dimension

Time convention. Both solvers’ internal radial ODE ($k_l = \sqrt{i\omega\mu_0\sigma}$) is solved natively in $e^{-i\omega t}$ (MINUS_IWT). PLUS_IWT output is produced by conjugating the finished field — conjugating a signal is exactly the transform between the two time conventions.

Harmonic normalization. `vsharm` (the shared Legendre/angular-derivative routine both solvers call) is fixed, always fully-normalized (FULLY_NORM): $Y_l^m = \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m e^{im\phi}$. SCHMIDT_SEMINORM output ($S_l^m = \sqrt{(2 - \delta_{m0}) \frac{(l-m)!}{(l+m)!}} P_l^m$, no $(2l+1)/(4\pi)$ growth factor) is produced by rescaling the SOURCE coefficients by $Y_{\text{schmidt}}/Y_{\text{full}} = \sqrt{4\pi(2 - \delta_{m0})/(2l+1)}$ before the solve.

Condon-Shortley phase. `vsharm`’s Y is fixed CS-included. The textbook identity $Y_l^{-m} = (-1)^m \bar{Y}_l^m$ is what makes the $\pm m$ coefficient-pairing reconstruction exact; requesting `condon_shortley=.false.` output flips every odd- m source coefficient by $(-1)^m$ before the solve.

Theta convention. COLAT_N2S (colatitude, north pole to south pole, increasing) vs. LAT_S2N (latitude, south to north, increasing) is the relabeling $\theta \rightarrow \pi - \theta$: the SAME physical field, viewed with the opposite array-index and angular-coordinate direction.

R convention. R_UP (radius from Earth’s center, increasing outward) vs. R_DOWN (depth below the surface, increasing inward) — again a relabeling of the same physical field, this time along the radial index.

Radial normalization. What `coeff(1,m)=1` means for the radial part of the field — see §4.2 for the exact per-degree amplitude formulas. This is the one dimension where the two solvers genuinely differ natively (S1=SURFACE, S2=MULTIPOLE); every other dimension the two solvers already agree on natively (§2).

2 Native conventions of the two solvers

Both solvers now share **identical** native bookkeeping in six of the seven dimensions: MINUS_IWT, FULLY_NORM, Condon-Shortley included, COLAT_N2S, R_UP, E -primary — exactly the SUNEGBERT2012 preset (§3). They differ natively in exactly one dimension, `radial_norm`: S1 is native SURFACE, S2 is native MULTIPOLE.

```
native_convention('S1') = SUNEGBERT2012 preset, but radial_norm = SURFACE
native_convention('S2') = SUNEGBERT2012 preset exactly (S2 + SUNEGBERT2012
is an identity)
```

History: why S1 originally differed, and why it doesn't any more

S1 (`field1d.f90`) was written in 2011 as a deliberate, faithful implementation of Kelbert (2006)'s own conventions — the Global3D Fortran code that is part of ModEM — and was used successfully with that code at the time. Two of Kelbert (2006)'s conventions are genuinely different physical choices from Sun & Egbert (2012)'s (which S2, written in 2026, follows natively):

1. **$\hat{\mathbf{r}}$ direction.** Kelbert (2006)'s radial unit vector points down (toward Earth's center); Sun & Egbert's points up (away from center). In the poloidal/toroidal field formulas

$$H = \frac{1}{r} \nabla_a \left(\frac{dT}{dr} \right) - \frac{\hat{\mathbf{r}}}{r^2} \nabla_a^2 T, \quad E = -i\omega\mu_0 \frac{\hat{\mathbf{r}}}{r} \times \nabla_a T, \quad (1)$$

∇_a / ∇_a^2 are purely angular and unaffected by this choice; H 's second term (which gives H_r) is $\hat{\mathbf{r}}$ -multiplied and flips sign under $\hat{\mathbf{r}} \rightarrow -\hat{\mathbf{r}}$, while H 's first term (H_θ , H_ϕ) has no $\hat{\mathbf{r}}$ in it and does not; E is entirely an $\hat{\mathbf{r}}$ -cross-product and flips as a whole. So the $\hat{\mathbf{r}}$ convention affects H_r , E_θ , E_ϕ together and leaves H_θ , H_ϕ untouched — this is the exact split originally observed between S1's raw output and S2's.

2. **Time-convention output.** Kelbert (2006)/Global3D natively outputs $e^{+i\omega t}$ fields, even though the internal radial solve is $e^{-i\omega t}$ natively (same as S2) — conjugating the assembled field is exactly how that conversion was made.

Both were genuine, deliberate design choices, not implementation errors. S1's raw/native output has since been re-pointed to match S2's convention directly (the $\hat{\mathbf{r}}$ -dependent formula signs were flipped in `field1d.f90`, and the time-convention conjugation was removed from the raw field assembly), so the two solvers now agree natively with no compensating sign anywhere in comparison code. Kelbert (2006)'s original convention is not lost — it is exactly the `KELBERT2006` preset below, reachable from either solver via `OUTPUT_CONVENTION='KELBERT2006'`.

The one bug genuinely fixed along the way (not a convention difference): the time-convention conjugation used to be baked directly into the $\pm m$ coefficient-pairing assembly loop, where it incorrectly re-conjugated the shared radial factor for paired terms but not for the $m = 0$ term — producing inconsistent output for sources that don't supply a complete $+m/-m$ pair. Moving the conjugation to an explicit, correctly-scoped post-assembly step (§4.3) fixed this.

3 The five named presets

Preset	time	norm	CS	theta	r	primary	modem	radial
KELBERT2006	PLUS	Schmidt	off	N2S	down	H	off	Surface
SUNEBERT2012	MINUS	Full	on	N2S	up	E	off	Multipole
EGBERTKELBERT2012	MINUS	Full	on	S2N	down	E	off	Surface
EGBERTKELBERT2012.MODEM	MINUS	Full	on	S2N	down	E	on	Surface
LWS	MINUS	Full	off	S2N	down	E	off	Potential

Table 2: The five named presets. “time” abbreviates `PLUS_IWT/MINUS_IWT`; “theta” `N2S=COLAT_N2S`, `S2N=LAT_S2N`.

Plus `OUTPUT_CONVENTION='NATIVE'`, which is not a fixed preset — it resolves dynamically to `native_convention(SOLVER)`, so `target==native` and every transform below is a mechanical no-op (a built-in round-trip-identity guarantee, not something that needs separate testing per preset).

- **KELBERT2006** — the original 2011 S1 output convention (§2). Named, not native to either solver’s current code, but always reachable.
- **SUNEBERT2012** — S2’s exact native convention. `SOLVER='S2'` with this preset is an identity: every transform is a no-op.
- **EGBERTKELBERT2012** — the practical “MT/ModEM-regional-like” convention: latitude increasing south-to-north (so plots read north-up without a manual flip) and depth increasing downward, matching how ModEM regional output is typically read. `condon_shortley=.true.` here was confirmed empirically against ModEM’s own `.prm` source files (they reproduce ModEM correctly when fed unscaled into either solver) — despite the historical ModEM solver this convention is named for never itself using spherical harmonics.
- **EGBERTKELBERT2012.MODEM** — identical bookkeeping to **EGBERTKELBERT2012**, plus the ModEM source-normalization value rescale (§4.4), for direct raw-field interoperability with ModEM.
- **LWS** — for NASA “Living With a Star” ionospheric-current (Sq-style) sources. Two deviations from S1/S2’s shared native bookkeeping, both testing hypotheses about how the LWS/MATLAB-SIEM source coefficients are actually authored: `radial_norm=POTENTIAL` (source coefficients are the classical external geomagnetic potential’s Gauss coefficients, not an amplitude of the solvers’ own toroidal potential T — a genuinely different formulation, not just a different scale) and `condon_shortley=.false..`

4 The transforms

Two are applied to the source coefficients before the solve; two are applied to the finished field arrays after.

4.1 Angular rescale (before the solve)

For each coefficient of degree l , order m (in the solvers’ own flat $m = 0, +1, -1, +2, -2, \dots$ ordering):

$$\text{if target\%condon_shortley} = \text{.false.} : \quad c *= (-1)^m \quad (2)$$

$$\text{if target\%harmonic_norm} = \text{SCHMIDT.SEMINORM} : \quad c *= \sqrt{4\pi(2 - \delta_{m0})/(2l + 1)} \quad (3)$$

Both ratios are computed relative to `vsharm`’s own fixed basis (fully-normalized, Condon-Shortley included — exactly **SUNEBERT2012**’s values), never relative to a nominal “source file convention.” This is what makes `target=SUNEBERT2012` a mechanically verifiable no-op (ratio = 1 for every l, m) and correctly reproduces the physical field for any other target, since feeding `rescaled_coeff = coeff * ratio` into the unchanged solver — which always combines coefficients with `vsharm`’s own fixed basis functions — gives the same result as if the target’s own basis functions had been used directly.

The Condon-Shortley flip direction and its structure are on solid footing (independently verified against a symbolic reference for $l \leq 3$, every m). The Schmidt/full normalization constant’s absolute value has not been independently cross-checked against an external reference (neither Sun & Egbert 2012 nor the project’s Python solver define a Schmidt-seminormalized basis at all) — flagged, not yet a concern for any currently-used preset since only **KELBERT2006** requests it and that preset is not load-bearing for any validated cross-check.

4.2 Radial rescale (before the solve)

Reconciles the ONE dimension where S1 and S2 differ natively. For each degree l :

$$\text{coeff} *= \frac{A(\text{target_radial}, l, R_0)}{A(\text{native_radial}(\text{solver}), l, R_0)} \quad (4)$$

where $A(\cdot, l, R_0)$ is defined relative to the MULTIPOLE baseline (a field expressed in convention X equals the same field in MULTIPOLE scaled by $A(X, l)$):

radial_norm	$A(l)$	Meaning
MULTIPOLE	1	S2 native. Sun & Egbert eq. (6): external term $T_l^{\text{ext}}(r) = \alpha_l^T (r/R_0)^{l+1}$, unit $\alpha_l^T = 1$.
SURFACE	$R_0^2/(l(l+1))$	S1 native. Unit external surface radial field ($H_r(R_0) = Y_l^m$ for coeff= 1).
DIMENSIONAL	R_0^{l+1}	Sun & Egbert's own text convention, coefficient of $r^{l+1} = 1$ (r in metres). Overflows double precision for $l \gtrsim 44$.
POTENTIAL	$R_0^2/(l+1)$	External scalar potential V (Gauss-coefficient) parameterization: $V = R_0 \sum_l (r/R_0)^l \varepsilon_l^m Y_l^m$, the classical geomagnetism/Sq-current convention.

Table 3: Per-degree radial amplitude $A(l)$ for each **radial_norm**.

POTENTIAL is derived from the insulating-atmosphere identity $H = -\nabla V$ matched against the solvers' own H formulas: $V_l(r) = -T_l'(r)$, giving $\varepsilon_l^m \propto \alpha_l^T/(l+1)$. MULTIPOLE/SURFACE/DIMENSIONAL all parameterize the same underlying quantity (an amplitude of the toroidal potential T), differing only in scale; POTENTIAL is a genuinely different formulation (the classical external potential V , used for ionospheric-current source coefficients), not just a rescaling.

4.3 Output-array transform (after the solve)

Applied to the finished H , E c-vectors:

1. **time**: if `native%time_convention` $\neq$ `target%time_convention`: conjugate every component of H and E .
2. **theta**: if `native%theta_convention` $\neq$ `target%theta_convention`: reverse the theta (2nd) index of every component array (each component's own extent — EDGE/FACE staggering gives $H_x/H_y/H_z$ each a different theta extent), and negate the theta components (H_y , E_y).
3. **r**: if `native%r_convention` $\neq$ `target%r_convention`: reverse the r (3rd) index of every component array, and negate the r components (H_z , E_z).

The index reversal and the component negation in steps 2–3 are a **matched pair, never applied separately**: the index reversal makes $\theta \rightarrow \pi - \theta$ (or the radial relabeling) a relabeling of the *same* physical field at the *same* physical point; the component negation accounts for the corresponding unit vector ($\hat{\theta}$ or $\hat{r}$) flipping direction under that relabeling. Applying only one gives a field that looks right at the equator (or at the reference radius) but is wrong everywhere else — this was verified explicitly with an off-equator physical sanity check ($H_\theta = \cos \theta$, confirming

the transformed value equals $-\cos\theta$ re-evaluated at the mirrored node, not just a sign flip at one point).

`primary_grid` is never transformed here — it is fixed by which raw solver ran (a solver produces H on EDGE and E on FACE, or vice versa, and that physical staggering doesn’t change after the fact). The `primary_grid` field on a *target* preset is descriptive only; the driver always sets the actual `primary_grid` flag from the active solver’s own native value (§5), never from the target convention.

4.4 ModEM source-normalization rescale (after §4.3, opt-in)

Applied only when `target%modem_normalize = .true.` (currently `EGBERTKELBERT2012_MODEM`). Rescales every component of H and E by the single complex factor

$$\text{factor} = \frac{1}{i\omega\mu_0 G}, \quad G = \frac{3}{2}\sqrt{\frac{3}{4\pi}} d_{\text{air}}, \quad (5)$$

where d_{air} is the air-column thickness (grid-top radius minus Earth radius, metres) and $\omega = 2\pi/\text{period}$. This converts global1d’s toroidal-potential (“unit external multipole”) source normalization into ModEM’s boundary-E-field (“unit surface E ”) normalization — the two differ by exactly $c = i\omega\mu_0 G$, the potential-vs-field factor of Faraday’s law (full derivation: [docs/source_normalization.md/.pdf](#)). Applying the identical factor to E and H preserves $Z = E/H$, so no physics changes, only the (otherwise arbitrary) source-amplitude scale. Validated against Cartesian ModEM to 0.2–0.6% at all tested periods.

5 FWD1D.f90 wiring

Three orthogonal top-level parameters:

```
character(len=20) :: SOLVER           = 'S1'      ! numerics only: 'S1' or 'S2'
character(len=24) :: OUTPUT_CONVENTION = 'LWS'    ! desired output bookkeeping
character(len=10) :: OUTPUT_FORMAT    = 'MODEM'  ! serialization: MODEM|GLOBAL|OFF
```

Per run:

1. `native = native_convention(SOLVER).`
2. `target_conv = native` if `OUTPUT_CONVENTION='NATIVE'`, else `get_convention(OUTPUT_CONVENTION).`
3. `primary_grid = native%primary_grid` — **always** the active solver’s own native staggering, never the target preset’s `primary_grid` field (descriptive only, per §4.3).
4. `eff_radial = RADIAL_SURFACE` if `target_conv%modem_normalize`, else `target_conv%radial_norm` (the ModEM rescale already fixes the final absolute scale, so a separate radial target would be redundant).
5. Per period/mode block: `rescale_source_coeffs` (§4.1) → `rescale_source_radial` (§4.2, `eff_radial`) → run the selected solver on the rescaled coefficients (unchanged solver code) → `apply_output_convention` (§4.3) → `apply_modem_normalization` if requested (§4.4).
6. Write output, tagging filenames and the file header with the solver name and `OUTPUT_CONVENTION` name so different solver/convention combinations never collide.

Current defaults, and a documentation note. As of this writing, `OUTPUT_CONVENTION='LWS'` is the actual compiled default (set for the active LWS/ionospheric-source project work). `FWD1D.f90`'s own inline comment above the parameter still marks `'EGBERTKELBERT2012_MODEM'` (DEFAULT) — that comment is stale relative to the parameter's actual current value and should be updated to match whichever convention is genuinely the default at the time it's read.

`OUTPUT_FORMAT='MODEM'` requires `primary_grid='E'` (both solvers' native default) since MoDEM's own `.esoln` format is always EDGE-staggered E .

6 Implementation guide: extending or repeating this work

For a human or AI engineer adding a new convention, a new dimension, or applying this pattern to a different solver pair.

6.1 Adding a new named preset

Presets are pure data — no solver code changes are ever needed. Add a new `type(output_convention_t)`, `parameter` with all seven dimension values plus `modem_normalize` and `radial_norm`, and add a `case` to `get_convention()`. That's it: every transform reads the preset's fields generically, so a new preset is usable everywhere immediately.

6.2 Adding a new convention dimension

1. Add the field to `output_convention_t`, and add its value to **every** existing preset (including `KELBERT2006`, `SUNEGBERT2012`, etc.) so their meaning doesn't silently change.
2. Decide which side of the solve it belongs on: does it affect what the **source coefficients** mean (extend `rescale_source_coeffs/rescale_source_radial`, before the solve), or does it affect the **output array**'s indexing or component sign (extend `apply_output_convention`, after the solve)? Keep this distinction — conflating the two makes the round-trip-identity guarantee (§3, `OUTPUT_CONVENTION='NATIVE'`) fail silently for one solver while passing for the other, since the solvers' own native coefficient/array conventions can differ independently on each side.
3. If the new dimension involves an array-index reversal (like `theta` or `r`), always pair it with the matching component-sign negation (§4.3) — never one without the other.
4. Add a self-consistency test (§6.3) before trusting the new dimension in a real comparison.

6.3 Verifying a change is safe

- **Angular rescale correctness:** hand-compute the $(-1)^m$ and Schmidt/full ratios for a few (l, m) including $m = 0$ (the easy-to-miss $(2 - \delta_{m0})$ case), and assert `rescale_source_coeffs` matches, for every preset.
- **Index-reversal correctness:** build a `cvector` on both `EDGE` and `FACE` with a known synthetic pattern, apply the reversal+negation, and check both (a) the index reversal itself (each component's own extent, not a shared `N` — `EDGE/FACE` staggering gives different components different extents along the reversed axis) and (b) a physical sanity check at an OFF-reference point (not just the equator/reference radius, where a sign-only bug can hide).

- **Solver agreement:** with the two solvers’ remaining native difference (`radial_norm`) reconciled, running both solvers under the SAME `OUTPUT_CONVENTION` must give identical output, to numerical precision — a single real ratio of 1, angle 0° , for every nonzero component of a representative test source.
- **Round-trip identity:** `OUTPUT_CONVENTION='NATIVE'` must reproduce each solver’s raw output bit-for-bit, for every solver — this is a structural guarantee (`target==native` makes every `if` branch in every transform false), not something that can silently break unless a transform is miswired to run unconditionally.
- **External cross-check:** compare against an independent reference (closed-form solution, or another code’s output) under a convention whose relationship to that reference is known — this is the only check that catches an error shared by both solvers (a bug in the transform layer itself, or in the reference relationship), which the solver-agreement check above cannot.

6.4 General lessons for repeating this pattern on a different solver pair

- **Separate “what does a source coefficient mean” (angular norm, Condon-Shortley, radial amplitude scale) from “what does the output array mean” (time convention, index direction, component sign)** — before vs. after the solve is the natural dividing line, and keeping it strict is what makes the round-trip-identity guarantee trustworthy.
- **Never bake a value-changing operation (like a time-convention conjugation) directly into a coefficient-reconstruction loop** that has its own internal structure (e.g. $\pm m$ pairing) — if the operation needs to apply to the whole assembled result but the loop’s own logic only naturally covers part of it (e.g. paired terms but not an unpaired $m = 0$ term), the two will interact incorrectly for inputs that don’t exercise the loop’s full intended structure. Keep such operations as a separate, explicit, always-correctly-scoped post-processing step instead.
- **Keep the convention module dependency-free of the solvers:** solvers and drivers depend on the convention type/transforms, never the reverse. This keeps every transform testable in isolation and keeps the solvers’ own numerics untouched by convention concerns entirely.
- **Make every convention a plain data value (a preset), not a code path.** A new convention should never require new solver code or new `if SOLVER==...` branches anywhere outside the driver’s convention-resolution step — if it does, the dimension probably needs to move from “output bookkeeping” into “solver behavior,” which is a different kind of change.